CAMPUS CAFÉ ORDERING SYSTEM

Java Object-Oriented Programming Laboratory

Name: Dangculos, Aschantal Myka B. **Section:** ITS-202-E

Date: November 10, 2026

Exercise Title: Campus Café Ordering System

I. Concept Explanation

1. Classes and Objects

Classes organize the program into different parts, such as Customer, Order, Beverage, Pastry, and Meal. Objects are created in the main program to represent customers, menu items, and orders.

2. Encapsulation

Encapsulation is used by making the fields private and accessing them through getters and setters. This helps protect the data and prevents direct access to it.

3. Inheritance

Inheritance is used through the MenuItem class, which serves as the parent class of Beverage, Pastry, and Meal. These subclasses inherit common properties and methods from the parent class.

4. Polymorphism

Polymorphism is applied through method overriding. Beverage, Pastry, and Meal provide their own implementations of methods such as computePrice() and getDescription().

5. Abstraction

Abstraction is implemented using the abstract MenuItem class and its abstract computePrice() method. It provides a common structure for all menu items without requiring the parent class to define the exact price calculation.

6. Constructor Overloading

Constructor overloading is used in the Customer class, which has three constructors with different parameters. This allows customers to be created in different ways.

7. Method Overloading

Method overloading is applied in the addPoints() method of Customer and the computeTotal() method of Order. These methods have different parameter lists to perform related operations.

II. Program Output

The screenshot below shows the actual output of the Campus Café Ordering System. It displays two customer orders, item prices, subtotals, discounts or vouchers, final totals, loyalty points, and validation error messages.

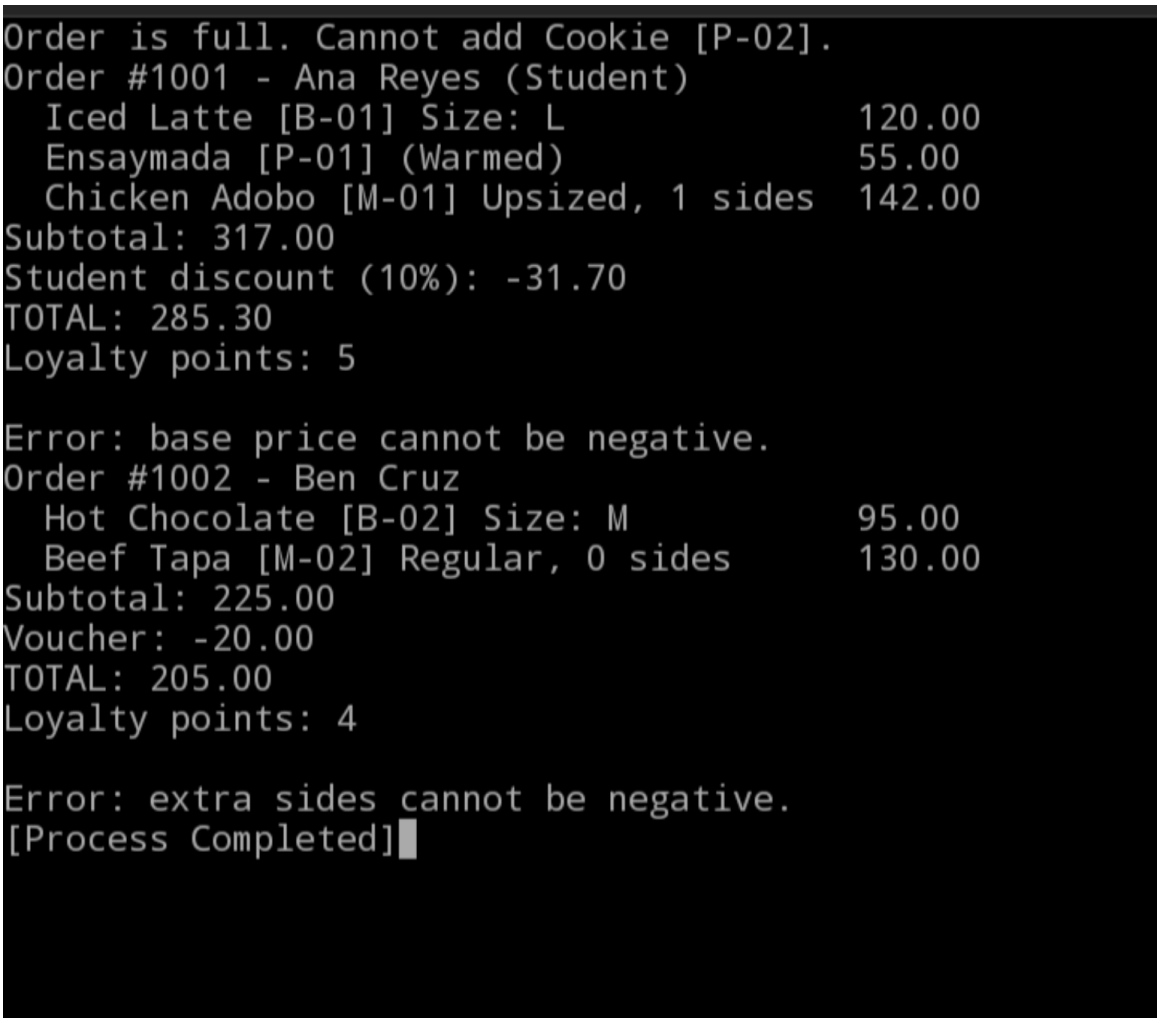


Figure 1. Actual program output showing Order #1001 and Order #1002.

III. Challenges Encountered and Solutions

While developing the Campus Café Ordering System, we encountered challenges in understanding how the different OOP concepts work together. We also experienced difficulties with price calculations, method overriding, constructor overloading, and applying discounts correctly. To solve these problems, we reviewed the instructions, checked the code, corrected errors, and tested the program several times. Through these activities, we gained a better understanding of Java programming and how OOP concepts are used in a real-world application.

IV. Conclusion

Overall, developing the Campus Café Ordering System helped us understand and apply the basic concepts of Object-Oriented Programming in Java. It also improved our skills in writing, organizing, debugging, and testing programs.